



Complemento Práctico: Orden y Búsqueda Universal

En la actividad anterior aprendimos a crear cajas genéricas (Sorteador<T>). Ahora daremos un paso más allá:

Comportamiento Genérico. ¿Cómo le enseñamos a Java a ordenar una lista de "Personas"? ¿Por edad? ¿Por apellido? ¿Y si quiero ordenar "Autos" por patente?

En esta unidad dominaremos las interfaces Comparable y Comparator, las herramientas estándar de la industria para ordenar colecciones. Finalmente, aplicaremos todo lo aprendido para resolver un problema clásico: dejar de escribir el mismo método buscarPorId una y otra vez, creando un **Buscador Genérico** capaz de encontrar cualquier objeto en cualquier lista.

Contenido

Interfaces Genéricas

El contrato Identificable<T> y la abstracción de claves

Orden Natural

Implementando Comparable<T> para definir comparaciones

Orden Flexible

Creando árbitros externos con Comparator<T>

Buscador Universal

Construcción del Buscador<T> genérico reutilizable

El Contrato de Identidad

En el video planteamos un problema fundamental: tenemos `Auto` (identificado por Patente, un `String`) y `Persona` (identificada por DNI, un `Long`). Queremos tratarlos a ambos como "cosas buscables". Para esto, creamos una interfaz que sea genérica no en el objeto, sino en su **clave de identificación**.

Esta abstracción es poderosa porque nos permite definir un contrato común para objetos que tienen naturalezas completamente diferentes. La clave puede ser cualquier tipo de dato: texto, números, incluso objetos compuestos. Lo importante es que todos cumplan el mismo protocolo de comunicación.

 **Concepto clave:** La interfaz `Identificable<T>` parametriza el tipo de la clave, no del objeto. Esto permite máxima flexibilidad en el diseño.

```
// T representa el TIPO de la clave (String, Long, Integer...)
public interface Identificable<T> {
    public T getId(); // Devuelve la clave
    public void setId(T id);
}
```

Implementación Híbrida

Clase Auto

Observen cómo cada clase "concreta" el tipo de dato de su ID. El Auto usa String para representar su patente, ya que las patentes son códigos alfanuméricos.

```
public class Auto
    implements Identificable<String>
{

    private String patente;

    @Override
    public String getId() {
        return patente;
    }

    @Override
    public void setId(String id) {
        this.patente = id;
    }
}
```

Clase Persona

La Persona usa Long para su DNI, ya que los documentos son números enteros largos. Mismo contrato, diferente implementación.

```
public class Persona
    implements Identificable<Long>
{

    private Long dni;

    @Override
    public Long getId() {
        return dni;
    }

    @Override
    public void setId(Long id) {
        this.dni = id;
    }
}
```

Esta flexibilidad es el corazón de la programación genérica: un mismo contrato se adapta a necesidades específicas sin perder su esencia universal.

"Yo sé compararme contra otro"

Java no sabe ordenar objetos por arte de magia. Si le dices

`Collections.sort(listaPersonas)`, Java preguntará: "*¿Quién va primero? ¿Juan o Ana?*".

Para responder esto, la clase debe implementar la interfaz `Comparable<T>`. Esto define el **Orden Natural** (el orden por defecto de un objeto).

Piensa en `Comparable` como darle a tu clase la capacidad intrínseca de saber dónde posicionarse en una fila. Es como si cada objeto llevara incorporado un sensor que le permite decir: "yo voy antes que ese" o "yo voy después de aquel". Esta es una capacidad *interna* del objeto.

1

Valor Negativo

Yo voy **antes** que el otro objeto

2

Valor Cero

Somos **iguales** en el ordenamiento

3

Valor Positivo

Yo voy **después** que el otro objeto

Implementación de Comparable

Veamos cómo una clase `Persona` implementa su orden natural basándose en el apellido. La belleza de este diseño es que podemos delegar la lógica de comparación a tipos que ya saben compararse, como los Strings.

```
public class Persona implements Comparable<Persona> {  
    private String nombre;  
    private String apellido;  
    private int edad;  
  
    @Override  
    public int compareTo(Persona otraPersona) {  
        // Delegamos la lógica a los Strings  
        // (que ya saben ordenarse alfabéticamente)  
        return this.apellido.compareTo(otraPersona.apellido);  
    }  
  
    // ... resto de la clase ...  
}
```

Uso Automático

Una vez implementado `Comparable`, podemos usar métodos de ordenamiento sin especificar criterios:

```
List<Persona> lista = new ArrayList<>()  
// ... agregar personas ...  
  
Collections.sort(lista);  
// ¡Ordenado por apellido automáticamente!
```

Ventajas

- Ordenamiento transparente y natural
- Compatible con todas las APIs de Java
- Define el orden "oficial" de la clase
- Código más limpio y expresivo

"Un árbitro externo decide"

¿Qué pasa si *también* quiero ordenar por edad? No puedo tener dos métodos `compareTo` en la misma clase. Aquí entra `Comparator<T>`. Es una clase externa ("árbitro") que toma dos objetos y decide cuál gana.

Mientras que `Comparable` es una capacidad intrínseca del objeto (como el color de ojos de una persona), `Comparator` es un criterio externo que podemos aplicar cuando queramos (como decidir ordenar personas por altura, por edad, o por nombre). Esto nos da una flexibilidad increíble.

1

Un Orden Natural

Solo un `compareTo()` por clase

2

Infinitos Comparadores

Múltiples criterios sin modificar la clase

Creando Comparadores Personalizados

Podemos crear tantos comparadores como criterios de ordenamiento necesitemos. Cada uno es una clase independiente que implementa la interfaz `Comparator<T>`.

Comparador por Edad

```
public class OrdenPorEdad
    implements Comparator<Persona> {

    @Override
    public int compare(Persona p1,
        Persona p2) {
        // Truco matemático: resta
        // 20 - 30 = -10 (negativo)
        // p1 va primero
        return p1.getEdad()
            - p2.getEdad();
    }
}
```

Comparador por Nombre

```
public class OrdenPorNombre
    implements Comparator<Persona> {

    @Override
    public int compare(Persona p1,
        Persona p2) {
        return p1.getNombre()
            .compareTo(p2.getNombre());
    }
}
```

❏ **Uso en código:** `lista.sort(new OrdenPorEdad());` o `lista.sort(new OrdenPorNombre());` – mismo método, diferentes criterios.

La ventaja estratégica es clara: podemos agregar nuevos criterios de ordenamiento sin tocar la clase `Persona`. Esto respeta el Principio Abierto/Cerrado: abierto a extensión, cerrado a modificación.

El Buscador Genérico: Fin del Copy & Paste

El punto culminante de la unidad. Normalmente escribimos un método `buscarAuto(String patente)` y otro `buscarPersona(Long dni)`. ¡Es el mismo código repetido una y otra vez! Usando **Genéricos + Bounded Wildcards**, creamos un solo método para todo.

El desafío técnico es fascinante: necesitamos un método que reciba una lista de "Cosas" (T) y una clave (K), pero con una condición estricta: esas "Cosas" deben saber identificarse. Aquí es donde la interfaz `Identificable` que creamos al principio cobra todo su sentido.

```
public class Buscador<T extends Identificable<K>, K> {  
  
    public T buscar(Collection<T> elementos, K idBuscado) {  
        for (T elemento : elementos) {  
            // Gracias a la interfaz, puedo llamar a getId()  
            if (elemento.getId().equals(idBuscado)) {  
                return elemento; // ¡Encontrado!  
            }  
        }  
        return null; // No estaba en la colección  
    }  
}
```

1

Método Universal

Reemplaza decenas de
métodos específicos

100%

Type-Safe

Seguridad de tipos en tiempo
de compilación

0

Código Duplicado

Elimina la repetición
innecesaria

Resumen y Conclusiones

Hemos alcanzado un nivel de abstracción profesional que se usa en sistemas reales de producción. Estas técnicas no son solo ejercicios académicos: son las herramientas que separan a un programador principiante de uno experimentado.

Interfaces Genéricas

Permitimos que Identificable se adapte a claves numéricas o de texto según la necesidad de cada clase, creando contratos flexibles.

Ordenamiento Dual

Comparable: La clase se ordena a sí misma (Orden Natural).
Comparator: Una clase externa ordena a otras (Múltiples Criterios).

Algoritmos Reutilizables

Creamos un Buscador que no depende de tipos concretos, sino de **capacidades**, ahorrando cientos de líneas duplicadas.

En sistemas grandes, estas técnicas marcan la diferencia entre código mantenible y código caótico. La próxima vez que te encuentres copiando y pegando un método, pregúntate: "*¿Puedo abstraer esto con genéricos?*". La respuesta, casi siempre, es sí.

"El mejor código es el que no tienes que escribir dos veces." — Principio de Reutilización